Seaborn Lab Manual

Experiment 1: Introduction to Seaborn and Statistical Plotting

Question: Load the built-in `tips` dataset and display its first rows and structure. Create a scatter plot of `total_bill` against `tip` in which the colour shows the meal time, the marker style shows whether the customer smokes, and the marker size shows the party size. Then create a line plot of the average tip for each party size, with a separate line for each gender.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Apply a clean default theme
sns.set_theme(style="whitegrid")

# Load the built-in dataset
tips = sns.load_dataset("tips")
print(tips.head())
tips.info()

# Scatter plot with hue, style and size
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=tips,
    x="total_bill",
    y="tip",
    hue="time",
    style="smoker",
    size="size",
    sizes=(30, 200)
)
plt.title("Total Bill vs Tip")
plt.xlabel("Total Bill ($)")
plt.ylabel("Tip ($)")
plt.show()

# Line plot: average tip by party size for each gender
plt.figure(figsize=(8, 5))
sns.lineplot(
```

```
    data=tips,
    x="size",
    y="tip",
    hue="sex",
    marker="o"
)
plt.title("Average Tip by Party Size")
plt.xlabel("Party Size")
plt.ylabel("Average Tip ($)")
plt.show()
```

Explanation:

- `sns.set_theme()` applies Seaborn's styling to every plot that follows, so no per-plot styling is needed.
- `sns.load_dataset("tips")` returns a pandas DataFrame, so `head()` and `info()` work as usual. Each row is one restaurant bill.
- `scatterplot()` takes the DataFrame through `data` and column names through `x` and `y`. Seaborn picks the colours and builds the legend itself.
- `hue` colours points by a column, `style` changes the marker shape, and `size` scales the marker. Together they show four variables in one plot.
- `lineplot()` groups rows that share the same `x` value and draws the mean `tip`. The shaded band around each line is a 95% confidence interval, which is something Matplotlib's `plot()` does not give you.
- `hue="sex"` draws one line per gender, which makes it easy to compare the two groups.

Experiment 2: Data Distribution Visualization

Question: Using the built-in `penguins` dataset, plot the distribution of `flipper_length_mm` as a histogram with a KDE curve. Compare the distribution of `body_mass_g` across the three species using overlapping KDE plots. Draw a stacked histogram of `bill_length_mm` by species. Finally, use `displot()` to draw one KDE panel per island, coloured by species. Calculate the skewness of `body_mass_g` and state whether the distribution is skewed.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")
```

```
penguins = sns.load_dataset("penguins")

# Summary statistics and skewness
print(penguins.describe())
print("Skewness of body mass:", penguins["body_mass_g"].skew())

# 1. Histogram with KDE curve
plt.figure(figsize=(8, 5))
sns.histplot(data=penguins, x="flipper_length_mm", bins=20, kde=True)
plt.title("Distribution of Flipper Length")
plt.show()

# 2. Overlapping KDE plots for each species
plt.figure(figsize=(8, 5))
sns.kdeplot(
    data=penguins,
    x="body_mass_g",
    hue="species",
    fill=True,
    common_norm=False,
    alpha=0.4
)
plt.title("Body Mass Distribution by Species")
plt.show()

# 3. Stacked histogram by species
plt.figure(figsize=(8, 5))
sns.histplot(
    data=penguins,
    x="bill_length_mm",
    hue="species",
    multiple="stack"
)
plt.title("Bill Length by Species (Stacked)")
plt.show()

# 4. Figure-level plot: one KDE panel per island
sns.displot(
    data=penguins,
    x="flipper_length_mm",
    hue="species",
    col="island",
```

```
    kind="kde",
    fill=True
)
plt.show()
```

Explanation:

- `histplot()` splits the data into bins and counts the values in each. `bins=20` sets the bin count, and `kde=True` adds a smooth curve over the bars.
- `skew()` measures asymmetry. A value near 0 means a roughly symmetric distribution. A positive value means a longer tail on the right, and a negative value means a longer tail on the left.
- `kdeplot()` draws a smoothed density curve instead of bars. `fill=True` shades the area under it, which helps when curves overlap.
- `hue="species"` draws one curve per species. `common_norm=False` scales each curve on its own, so a species with fewer penguins is not drawn smaller than the others.
- `multiple="stack"` stacks the species on top of each other in each bin, which shows how each species contributes to the total.
- `displot()` is a figure-level function. It creates its own figure, so `plt.figure()` is not needed before it. `col="island"` produces one panel per island, and `kind="kde"` switches it from a histogram to a density plot.
- Penguins with missing values are left out of each plot automatically, so no manual cleaning is needed here.

Experiment 3: Categorical Data Visualization

Question: Using the `tips` dataset, create four plots to compare categories. First, use `countplot()` to show the number of bills for each day. Then use `barplot()` to show the average `total_bill` for each day, split by `sex`. Next, use `boxplot()` to compare the spread of `tip` across days. Finally, use `violinplot()` to compare the distribution of `total_bill` for lunch and dinner, split by `smoker`.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")
tips = sns.load_dataset("tips")
```

```
# 1. Count of bills per day
plt.figure(figsize=(7, 5))
sns.countplot(data=tips, x="day", order=["Thur", "Fri", "Sat", "Sun"])
plt.title("Number of Bills per Day")
plt.show()

# 2. Average total bill per day, split by gender
plt.figure(figsize=(7, 5))
sns.barplot(data=tips, x="day", y="total_bill", hue="sex")
plt.title("Average Total Bill per Day")
plt.show()

# 3. Box plot of tips across days
plt.figure(figsize=(7, 5))
sns.boxplot(data=tips, x="day", y="tip")
plt.title("Tip Distribution per Day")
plt.show()

# 4. Violin plot of total bill by meal time, split by smoker
plt.figure(figsize=(7, 5))
sns.violinplot(data=tips, x="time", y="total_bill", hue="smoker", split=True)
plt.title("Total Bill by Meal Time and Smoker Status")
plt.show()
```

Explanation:

- `countplot()` counts the rows in each category, so only `x` is needed. `order` fixes the sequence of the bars, which otherwise follows the order in which the days appear in the data.
- `barplot()` shows the mean of `y` for each category. The thin black line on each bar is a 95% confidence interval for that mean. `hue="sex"` places a second bar next to the first for each gender.
- `boxplot()` summarises the spread of a variable. The box covers the middle 50% of values, the line inside it is the median, and the points beyond the whiskers are outliers.
- `violinplot()` combines a box plot with a KDE. The width of each shape shows how many values lie at that level, so it reveals features such as two peaks that a box plot hides.
- `split=True` draws one half of each violin per `hue` group, which saves space when there are only two groups.

- Use a count plot for frequencies, a bar plot for averages, and a box or violin plot when the spread of the data matters.

Experiment 4: Relationship Analysis Using Seaborn

Question: Using the `penguins` dataset, plot `flipper_length_mm` against `body_mass_g` as a scatter plot coloured by species, with marker style showing sex. Draw a line plot of the average body mass at each flipper length for every species. Then use `relplot()` to split the scatter plot of `bill_length_mm` against `bill_depth_mm` into one panel per island, with colour showing species and marker style showing sex.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")
penguins = sns.load_dataset("penguins")

# 1. Scatter plot with hue and style
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=penguins,
    x="flipper_length_mm",
    y="body_mass_g",
    hue="species",
    style="sex"
)
plt.title("Flipper Length vs Body Mass")
plt.show()

# 2. Line plot of average body mass at each flipper length
plt.figure(figsize=(8, 5))
sns.lineplot(
    data=penguins,
    x="flipper_length_mm",
    y="body_mass_g",
    hue="species",
    errorbar=None
)
plt.title("Average Body Mass by Flipper Length")
plt.show()
```

```
# 3. Figure-level plot: one panel per island
sns.relplot(
    data=penguins,
    x="bill_length_mm",
    y="bill_depth_mm",
    hue="species",
    style="sex",
    col="island",
    kind="scatter",
    height=4,
    aspect=0.9
)
plt.show()
```

Explanation:

- The first scatter plot shows that body mass rises with flipper length. Colouring by species shows that the three species form separate clusters along that trend.
- `style="sex"` uses a different marker for each sex. Penguins with no recorded sex are left out of this plot.
- `lineplot()` averages `body_mass_g` for every distinct flipper length and draws the result as a line. `errorbar=None` removes the confidence band so the lines stay readable. The lines are jagged where few penguins share a flipper length, so read them as a trend and not as exact values.
- `relplot()` is the figure-level version of `scatterplot()` and `lineplot()`. The `kind` argument chooses which of the two it draws.
- `col="island"` creates one panel per island on a shared set of axes, so the islands can be compared directly. Adding `row=` would split the data a second way.
- `height` and `aspect` set the size of each panel, instead of `plt.figure(figsize=...)`, which has no effect on figure-level plots.
- Within each species, bill length and bill depth rise together, even though the pooled data trend the opposite way. Colouring and splitting by `hue` and `col` is what reveals this.

Experiment 5: Correlation Analysis and Heatmaps

Question: Using the `penguins` dataset, calculate the correlation matrix of the four numerical measurements. Display it as an annotated heatmap with a diverging colour map centred at zero. Then draw a second heatmap that shows only the lower triangle of the

matrix, so that each pair of variables appears once. Identify the strongest positive and the negative correlations.

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="white")
penguins = sns.load_dataset("penguins")

# Select the numerical columns and compute the correlation matrix
numeric = penguins[["bill_length_mm", "bill_depth_mm",
                    "flipper_length_mm", "body_mass_g"]]
corr = numeric.corr()
print(corr.round(2))

# 1. Full annotated heatmap
plt.figure(figsize=(7, 6))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    center=0,
    linewidths=0.5
)
plt.title("Correlation Matrix of Penguin Measurements")
plt.show()

# 2. Lower-triangle heatmap
mask = np.triu(np.ones_like(corr, dtype=bool))
plt.figure(figsize=(7, 6))
sns.heatmap(
    corr,
    mask=mask,
    annot=True,
    fmt=".2f",
    cmap="viridis",
    vmin=-1,
    vmax=1,
    square=True,
```

```
    cbar_kws={"label": "Correlation coefficient"}
)
plt.title("Lower-Triangle Correlation Heatmap")
plt.show()
```

Explanation:

- `corr()` computes the Pearson correlation coefficient for every pair of columns. Each value lies between -1 (perfect negative relationship) and +1 (perfect positive relationship). Rows with a missing value are skipped for the pair of columns involved.
- `heatmap()` turns the matrix into a grid of coloured cells. `annot=True` writes the value in each cell, and `fmt=".2f"` rounds it to two decimal places.
- `cmap="coolwarm"` is a diverging colour map, which suits correlations because it uses one colour for positive values, another for negative values, and a neutral colour near zero. `vmin`, `vmax` and `center` fix the scale from -1 to 1 so the colours always have the same meaning.
- The diagonal is always 1 because every variable correlates perfectly with itself. The matrix is also symmetric, so the upper half repeats the lower half.
- `np.triu()` builds an upper-triangle mask, and passing it as `mask` hides those cells. `square=True` keeps the cells square, and `cbar_kws` labels the colour bar.
- In this dataset, flipper length and body mass have the strongest positive correlation, at about 0.87. Bill depth has a negative correlation with flipper length and with body mass.
- A correlation shows that two variables move together, not that one causes the other. The species grouping in Experiment 4 is one reason these values should be read with care.

Experiment 6: Pairwise Analysis and Customized Statistical Plots

Question: Using the built-in `iris` dataset, draw a pair plot of all four measurements coloured by species. Draw a joint plot of `sepal_length` and `petal_length` that also shows each variable's distribution. Use `FacetGrid` to draw one scatter plot of `petal_length` against `petal_width` for each species. Finally, place a box plot and a violin plot of `petal_length` side by side in one figure, using a custom palette, titles and axis labels.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="ticks")
iris = sns.load_dataset("iris")

# 1. Pair plot of all numerical columns
g = sns.pairplot(iris, hue="species", palette="Set2", diag_kind="kde")
g.figure.suptitle("Pairwise Relationships in the Iris Dataset", y=1.02)
plt.show()

# 2. Joint plot with marginal distributions
g = sns.jointplot(
    data=iris,
    x="sepal_length",
    y="petal_length",
    hue="species",
    palette="Set2"
)
g.figure.suptitle("Sepal Length vs Petal Length", y=1.02)
plt.show()

# 3. FacetGrid: one scatter plot per species
g = sns.FacetGrid(iris, col="species", height=4)
g.map_dataframe(sns.scatterplot, x="petal_length", y="petal_width")
g.set_axis_labels("Petal Length (cm)", "Petal Width (cm)")
g.set_titles(col_template="{col_name}")
g.figure.suptitle("Petal Size for Each Species", y=1.05)
plt.show()

# 4. Two plots side by side with a custom palette
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.boxplot(
    data=iris, x="species", y="petal_length",
    hue="species", palette="Set2", legend=False, ax=axes[0]
)
axes[0].set_title("Box Plot of Petal Length")
axes[0].set_xlabel("Species")
axes[0].set_ylabel("Petal Length (cm)")

sns.violinplot(
    data=iris, x="species", y="petal_length",
```

```
    hue="species", palette="Set2", legend=False, ax=axes[1]
)
axes[1].set_title("Violin Plot of Petal Length")
axes[1].set_xlabel("Species")
axes[1].set_ylabel("Petal Length (cm)")

fig.suptitle("Comparing Petal Length Across Species")
plt.tight_layout()
plt.show()
```

Explanation:

- `pairplot()` draws a grid in which every numerical column is plotted against every other one. Each off-diagonal cell is a scatter plot, and each diagonal cell shows the distribution of one variable (a KDE here, because of `diag_kind="kde"`). It gives a quick overview of the whole dataset.
- In the pair plot, setosa separates clearly from the other two species in the petal measurements, while versicolor and virginica overlap more.
- `jointplot()` combines a scatter plot in the centre with the distribution of each variable along the top and right edges. With `hue`, the margins are drawn as one KDE per species.
- `pairplot()` and `jointplot()` return grid objects, not axes. Titles for the whole figure go on `g.figure.suptitle()`, and `y` moves the title above the plot.
- `FacetGrid` splits the data into panels by the values of a column. `map_dataframe()` then applies one plotting function to every panel, with `x` and `y` given as column names.
- `set_axis_labels()` and `set_titles()` change the labels on all panels at once. `{col_name}` in the template is replaced by the value of that panel.
- `palette` selects a colour set for the whole figure. The box and violin plots use the same palette, so each species has the same colour in both.
- For a plot made with `plt.subplots()`, pass `ax=` to send a Seaborn plot to a particular panel, and use `set_title()`, `set_xlabel()` and `set_ylabel()` on that panel. `fig.suptitle()` adds one title above both.